Course 50.050/50.550
Advanced Algorithms

Week 5 – Lecture L05.01

SINGAPORE UNIVERSITY OF
TECHNOLOGY AND DESIGN

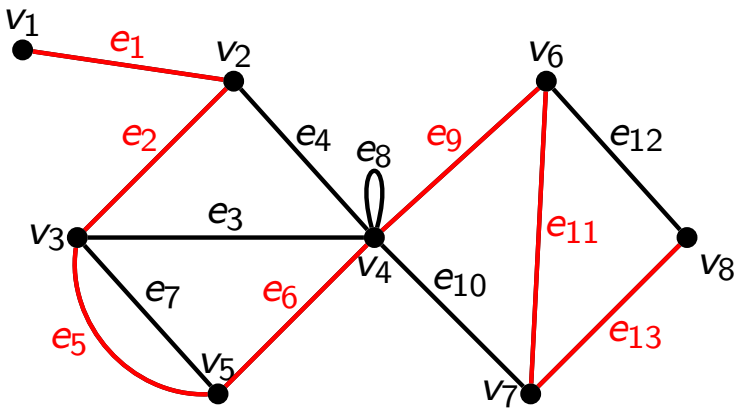
Outline of Lecture

- Walks, trails, paths
- Eulerian paths, Eulerian cycles
- Euler's theorem
- Introduction to Fleury's algorithm

Informal intuition for paths

Intuition: A path can be visualized as a contiguous sequence of edges joining some "start" vertex to some "end" vertex. We could follow the path to go from the "start" vertex to the "end" vertex.

- **Example:** A graph with a path depicted by the red edges.



- Imagine we have a **walk** starting from vertex v_1 and going towards vertex v_8 along the path.
  - After traversing through each edge in the path, we would reach a vertex, before we start traversing through another edge.
  - For our path, we would visit the following alternating sequence of vertices and edges:
    - $v_1, e_1, v_2, e_2, v_3, e_5, v_5, e_6, v_4, e_9, v_6, e_{11}, v_7, e_{13}, v_8$.

Graph-theoretic terminology for paths

Important Note: Terminology for paths is a complete mess!

- There is absolutely no single agreed-upon terminology and notation related to paths, and we would probably need **centuries** before the terminology becomes consistent across different textbooks
  - Or perhaps we will never ever get any standardization.
- Currently, there are two competing attempts for "standardization".
  - One "camp" has more computer scientists and more algorithms.
  - One "camp" has more mathematicians and more theorems.

Question: Edges only? Both vertices and edges? Or vertices only?

- Is a path an alternating sequence of vertices and edges?
- Do we want a path to be a sequence of edges?
- Or do we want a path to be a sequence of vertices?

Question: Any repeats allowed?

- Do we allow the "visited" edges in a path to repeat?
- Do we allow the "visited" vertices in a path to repeat?

Warning: When checking any online source or textbook, you should **always** check the definition of "path" being used!

What is a walk?

Definition: Let $G = (V, E, \phi)$ be a graph. A walk in G is a sequence $\mathbf{p} = (v_0, e_1, v_1, e_2, v_2, \ldots, e_k, v_k)$, whose entries alternate between vertices and edges, and whose first entry v_0 and last entry v_k are vertices, such that the edges $e_1, \ldots e_k$ satisfy the following:

- (If G is undirected): $\phi(e_i) = \{v_{i-1}, v_i\}$.
  - e_i could be a loop, in which case $\phi(e_i) = \{v_{i-1}\} = \{v_i\}$.
- (If G is directed): $\phi(e_i) = (v_{i-1}, v_i)$.

We say that $\mathbf{p}$ is a walk from v_0 to v_k. The number of edges in $\mathbf{p}$ is called the length of $\mathbf{p}$.

Note: The vertices and edges could repeat!

- Just like how in real life, we could walk back and forth, a walk in a graph may have repeated edges.

Note: A walk in a directed graph is sometimes called a directed walk.

Warning! Some authors instead define a walk as a sequence of edges, just like how we have defined a walk, but omitting the vertices visited from the sequence. Other authors instead define a walk as a sequence of vertices, just like how we have defined a walk, but omitting the edges from the sequence.

- This last definition (a walk as a sequence of vertices) is ambiguous for non-simple graphs, especially if we consider "walks without repeated edges".

What is a trail?

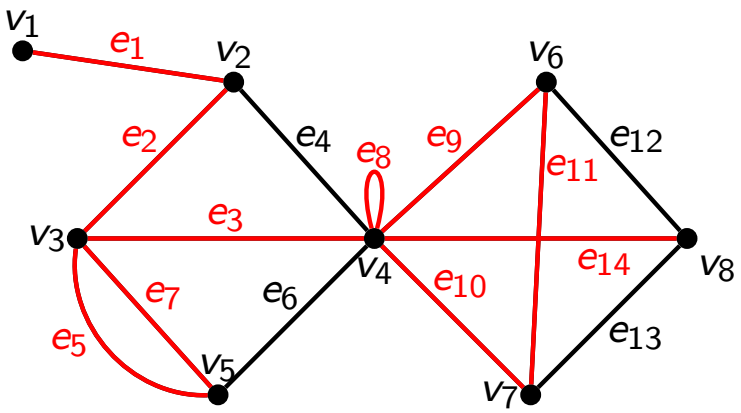
Definition: Let $G = (V, E, \phi)$ be a graph. A trail in G is a walk $\mathbf{p} = (v_0, e_1, v_1, \ldots, e_k, v_k)$ with **distinct** edges $e_1, \ldots e_k$.

Warning! Some authors instead define a trail as a sequence of edges, just like how we have defined a trail, but omitting the vertices visited from the sequence. Other authors instead define a trail as a sequence of vertices, just like how we have defined a trail, but omitting the edges from the sequence.

- This last definition (a trail as a sequence of vertices) is ambiguous for non-simple graphs, since it is not clear which edges are involved.

Note: In a trail, edges cannot repeat, but the vertices could repeat.

Example: A trail with repeated vertices.



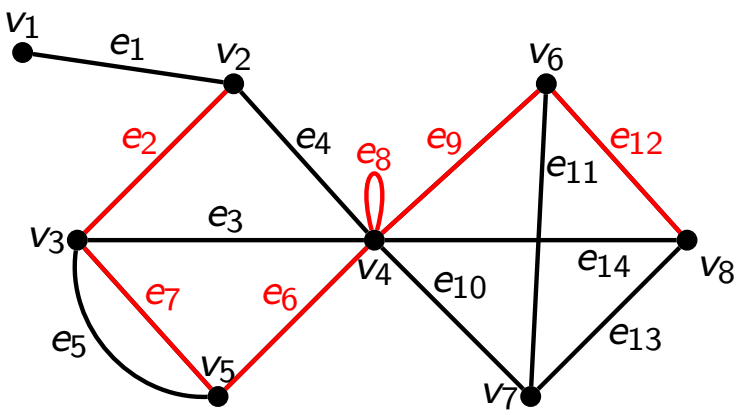
- $\mathbf{p} = (v_1, e_1, v_2, e_2, v_3, e_5, v_5, e_7, v_3, e_3, v_4, e_8, v_4, e_9, v_6, e_{11}, v_7, e_{10}, v_4, e_{14}, v_8)$ is a trail of length 10 from v_1 to v_8.

Open versus closed walks

Definition: Let $\mathbf{p}$ be a walk in a graph G from u to v.

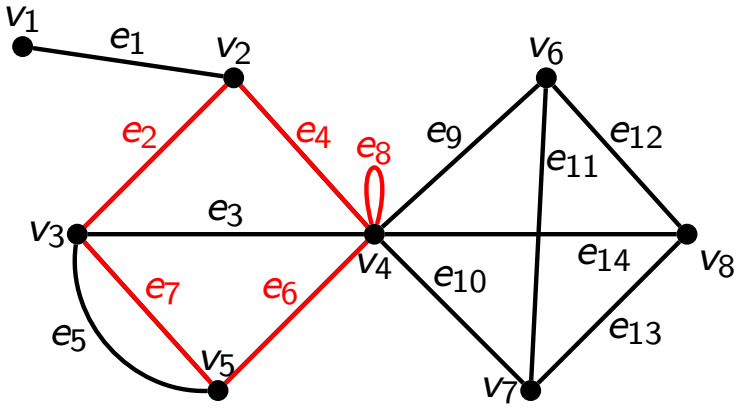
- If $u = v$ are the same vertex, then we call $\mathbf{p}$ a closed walk.
- If $u \neq v$ are distinct vertices, then we call $\mathbf{p}$ an open walk.

Example:



- $\mathbf{p} = (v_2, e_2, v_3, e_7, v_5, e_6, v_4, e_8, v_4, e_9, v_6, e_{12}, v_8)$ is an open trail from v_2 to v_8.

Example:



- $\mathbf{p} = (v_2, e_2, v_3, e_7, v_5, e_6, v_4, e_8, v_4, e_4, v_2)$ is a closed trail from v_2 to v_2.

Shortest walks

Definition: Let $G = (V, E, \phi)$ be a graph, and let u, v be vertices in V. A walk $\mathbf{p}$ in G from u to v is called a shortest walk from u to v if among all possible walks from u to v, the smallest possible length of the walk equals the length of $\mathbf{p}$.

- **Note:** There could be multiple shortest walks from u to v.
- **Note:** There is only one shortest walk from u to u, i.e. the walk (u) of length 0.
- **Fact:** A shortest walk $\mathbf{p} = (v_0, e_1, v_1, \ldots, e_k, v_k)$ cannot have any repeated vertices or any repeated edges.

Definition: Let $G = (V, E, \phi)$ be a graph, and let $u, v \in V$.

- (If G is undirected): If there exists a walk from u to v, then we say that u and v are connected, and we define the distance between u and v (or the distance from u to v) to be the length of any shortest walk from u to v.
- (If G is directed): If there exists a walk from u to v, then we say that u is connected to v, and we define the distance from u to v to be the length of any shortest walk from u to v.

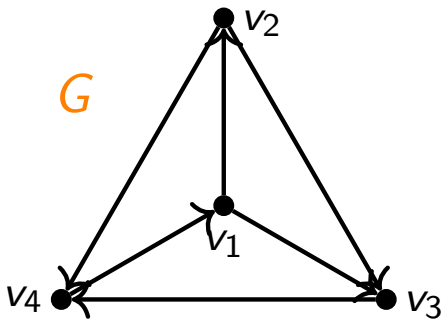
Number of walks with given length

Theorem: Let $G = (V, E)$ be a simple (undirected or directed) graph, and let A be the adjacency matrix for G.

- Let $k \in \mathbb{Z}^+$, suppose that $V = \{v_1, \ldots, v_n\}$ has n vertices, and let $1 \leq i, j \leq n$. Then the total number of walks of length k from v_i to v_j equals the (i, j)-th entry in the matrix product A^k.

Proof Idea: By induction on k.

Example: Graph G with adjacency matrix A.



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \quad A^3 = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 2 \end{bmatrix} \quad A^4 = \begin{bmatrix} 1 & 2 & 2 & 0 \\ 0 & 1 & 2 & 2 \\ 0 & 0 & 1 & 2 \\ 2 & 0 & 0 & 1 \end{bmatrix}$$

- There are 0 walks of length 3 from v_4 to v_1, but 2 walks of length 4 from v_4 to v_1.

Question: Recall Exercise 1 of Week 4's cohort class. We showed that if G is a simple **undirected** graph with adjacency matrix A, then the trace $\mathrm{tr}(A)$ equals twice the number of edges in G.

- How can you show this as a consequence of the theorem above?

What is a path?

WARNING! There are multiple **widely used** definitions for paths!

Based on **our definitions** for open/closed walks and open/closed trails, these are some of the definitions of paths you may encounter:

- Some define a "path" to be what we call a walk.
- Some define a "path" to be what we call a trail.
- Some define a "path" to be what we call an open trail.
- Some define a "path" to be what we call a walk with distinct vertices.
- Some define a "path" to be what we call a walk with distinct vertices, except possibly the first and last vertices.
- Some define a "path" to be what we call a trail with distinct vertices, except possibly the first and last vertices.

Warning: There is still the issue of whether a path is an alternating sequence of vertices and edges, or a sequence of edges, or a sequence of vertices.

Note: Since a **shortest walk** cannot have any repeated vertices or repeated edges, the notion of a "**shortest path**" would essentially have the same definition, for all of these six definitions of "path".

- **Warning:** For weighted graphs, "shortest path" are instead defined as paths with the smallest weight.
  - We will subsequently study weighted graphs in detail.

So what is a path??

We shall take a practical approach, and define "path" that is most suitable for graph algorithms involving paths.

- We shall use the "same path definition" from 50.004 Algorithms, when we covered shortest-paths algorithms.
  - Remember Dijkstra's algorithm? Bellman–Ford algorithm?

Definition: (**For 50.050 only**) Let $G = (V, E, \phi)$ be a graph. A path in G is a sequence of edges $\mathbf{p} = (e_1, e_2, \ldots, e_k)$ satisfying the following:

- There exists $k + 1$ vertices $v_0, v_1, \ldots, v_k$ such that the alternating sequence of vertices and edges $(v_0, e_1, v_1, e_2, \ldots, e_k, v_k)$ is a walk.
  - For convenience, we shall also use the notation $\langle v_0, v_1, \ldots, v_k \rangle$ to represent this same path $\mathbf{p}$ as a sequence of vertices.
  - We say that $\mathbf{p}$ is a path from v_0 to v_k. The number of edges in $\mathbf{p}$ is called the length of $\mathbf{p}$.

Warning: The notation $\langle v_0, v_1, \ldots, v_k \rangle$ comes from [CLRS] but this notation is non-standard!

Note: A path is NOT a subgraph! A path is a sequence of edges.

- We think of $\langle v_0, v_1, \ldots, v_k \rangle$ as a different "path representation", analogous to how there are different graph representations for the same graph.

More on path terminology

Intuition: Our 50.050 definition of "path" is essentially equivalent to our definition of "walk", but with two different representations. Suppose $(v_0, e_1, v_1, e_2, \ldots, e_k, v_k)$ is a walk in a graph.

- The subsequence $(e_1, e_2, \ldots, e_k)$ of the walk with the vertices omitted is what we call a "path".
- The vertices $v_0, v_1, \ldots, v_k$ that we have omitted, when written as "$\langle v_0, v_1, \ldots, v_k \rangle$" is another representation of this same path.

Note: Our 50.050 definition of "paths" allows for repeated edges and repeated vertices.

- The definition of "path" that many mathematicians use is either a "walk without repeated vertices", or a "walk without repeated vertices, except possibly for the first and last vertices".
  - Hence, for many authors, a walk without repeated edges is a trail, and a walk without repeated vertices is a path.

Question: Why introduce the definitions of "walk" and "trail"?

- The good thing about first defining "walk" and "trail" is that they have relatively more consistent definitions, unlike the mess for defining "path" (with multiple incompatible "path" definitions).

More on path terminology (continued)

Definition: (**For 50.050 only**) Let $G = (V, E, \phi)$ be a graph, and let $\mathbf{p} = (e_1, e_2, \ldots, e_k)$ be a path in G. A subpath of $\mathbf{p}$ is a contiguous subsequence $(e_i, e_{i+1}, \ldots, e_j)$ of $\mathbf{p}$.

- The edges in a subpath must be consecutive entries of $\mathbf{p}$.

Definition: (**For 50.050 only**) Let $G = (V, E, \phi)$ be a graph. A simple path in G is a sequence of edges $\mathbf{p} = (e_1, e_2, \ldots, e_k)$ that satisfies the following:

- There exist $k + 1$ **distinct** vertices $v_0, v_1, \ldots, v_k$ such that the alternating sequence $(v_0, e_1, v_1, e_2, \ldots, e_k, v_k)$ of vertices and edges is a walk.
  - Again for convenience, we shall use the notation $\langle v_0, v_1, \ldots, v_k \rangle$ to represent this same path $\mathbf{p}$ as a sequence of vertices.
  - We say that $\mathbf{p}$ is a path from v_0 to v_k. The number of edges in $\mathbf{p}$ is called the length of $\mathbf{p}$.

Warning: Some authors define "simple path" to mean what we call a trail (i.e. a walk with distinct edges), but as a sequence of edges, with vertices omitted.

- Some authors define "simple path" as a sequence of edges, while others define it as a sequence of vertices. For us, we shall use both representations.

What is a cycle?

WARNING! There are multiple **widely used** definitions for cycles!

Based on **our definitions** for open/closed walks and open/closed trails, these are some of the definitions of cycles you may encounter:

- Some define a "cycle" to be what we call a closed walk.
- Some define a "cycle" to be what we call a closed trail.
- Some define a "cycle" to be what we call a walk with distinct vertices, except for the first and last vertices, which are equal.

Warning: There is still the issue of whether a cycle is an alternating sequence of vertices and edges, or a sequence of edges, or a sequence of vertices.

- Some authors define "cycle" as a sequence of edges, while others define it as a sequence of vertices.

Note: Some authors use the terminology circuit or tour instead of cycle.

Important Note: No matter which of these definitions for "cycle" is used, a cycle in a graph G is NEVER a subgraph of G.

- Although a cycle graph C_n is a graph, a cycle is NOT a graph.
  - Recall: An n-cycle is a cycle graph with n vertices. Hence, a graph could contain an n-cycle as a subgraph, but a cycle in a graph is always a sequence (whether it is a sequence of alternating edges and vertices, or a sequence of edges, or a sequence of vertices).

So what is a cycle??

Recall: A closed walk in a graph G is a walk $(v_0, e_1, v_1, e_2, \ldots, e_k, v_k)$ such that $v_0 = v_k$ (i.e. the first and last vertices are the same).

- **Intuition:** A closed walk is a walk starting from some vertex v_0 that "closes" back to itself.

Definition: (**For 50.050 only**) Let $G = (V, E, \phi)$ be a graph.

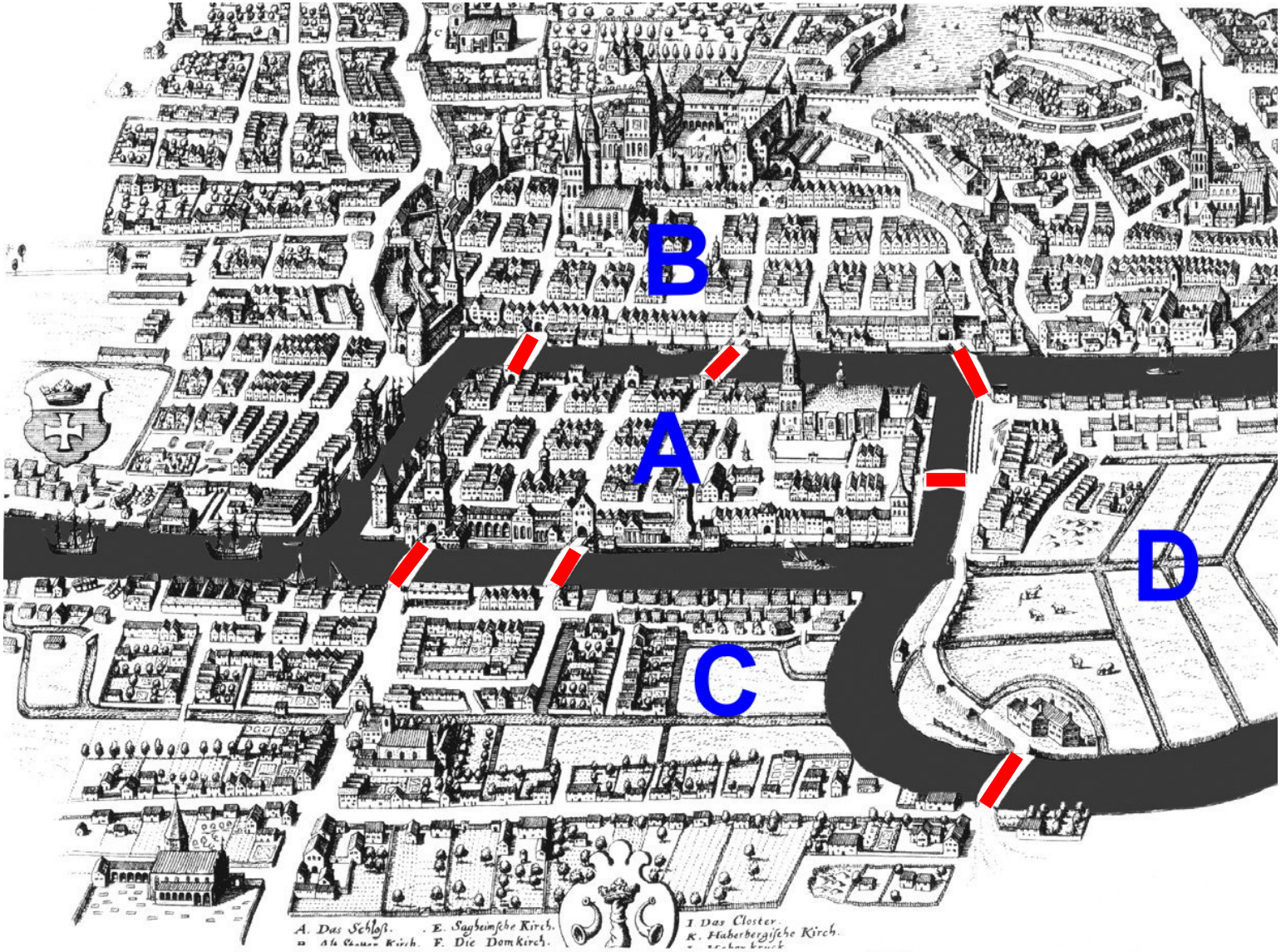
- A cycle in G is a path $\mathbf{p} = (e_1, \ldots, e_k)$ in G with at least one edge, for which there exist $k+1$ vertices $v_0, v_1, \ldots, v_k$ such that $(v_0, e_1, v_1, e_2, \ldots, e_k, v_k)$ is a closed walk, i.e. $v_0 = v_k$.
  - Equivalent notation: We could also write $\mathbf{p} = \langle v_0, v_1, \ldots, v_k \rangle$.
- A simple cycle in G is a cycle $\mathbf{p} = (e_1, \ldots, e_k)$ in G with at least one edge, for which there exist k **distinct** vertices $v_0, v_1, \ldots, v_{k-1}$ in G, such that $(v_0, e_1, v_1, e_2, \ldots, v_{k-1}, e_k, v_0)$ is a **closed** walk.
  - Equivalent notation: We could also write $\mathbf{p} = \langle v_0, v_1, \ldots, v_{k-1}, v_0 \rangle$.

Note: Some authors use "simple circuit" instead of "simple cycle".

Warning: Some authors define "simple cycle" to be what we call a closed trail (i.e. a closed walk with distinct edges), but as a sequence of edges, with vertices omitted, or as a sequence of vertices, with edges omitted.

Seven Bridges of Königsberg Problem

(Königsberg is today's Kaliningrad, located in Russia.)

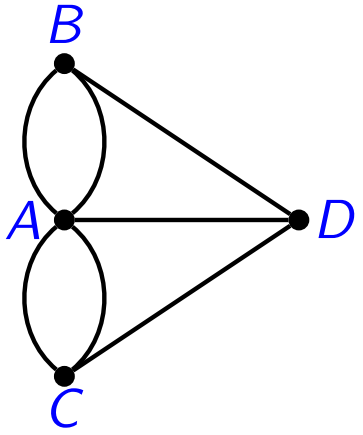


Problem: Is it possible to walk through the city and cross each of the seven bridges once and exactly once?

Eulerian path

Interesting Fact: Euler's solution (in 1736) to the Seven Bridges of Königsberg Problem is the first ever theorem in graph theory!

- The first graph ever considered was a non-simple graph!



Euler proved that it is **not possible** to walk through the city and cross each of the seven bridges once and exactly once.

- He introduced the idea of graphs, and the idea of paths in a graph.

Definition: Let $G = (V, E, \phi)$ be a graph. An Eulerian path in G is a **trail** in G that **contains all edges** in E.

- **Intuition:** An Eulerian path is a path with all edges traversed, with each edge traversed exactly once. Vertices could repeat.

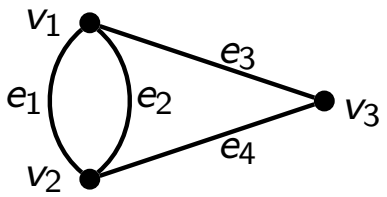
Using modern terminology, Euler proved that the above graph has no Eulerian path.

Eulerian cycles

Definition: Let $G = (V, E, \phi)$ be a graph. An Eulerian cycle in G is a **closed trail** in G that **contains all edges** in E.

- **Intuition:** An Eulerian cycle is a cycle with all edges traversed, with each edge traversed exactly once. Vertices could repeat.
- **Note:** It is possible for a graph to have an Eulerian path, but not an Eulerian cycle.

Example: A graph with an Eulerian path.



- Example of Eulerian path: $(v_1, e_1, v_2, e_2, v_1, e_3, v_3, e_4, v_2)$.
  - Recall that in 50.050, we define an Eulerian path to be a trail.
- **Note:** This graph has no Eulerian cycle.

Question: Can you think of a characterization for Eulerian cycles in terms of a **graph invariant**? What about Eulerian paths?

- A characterization for an object "type" X is a set of conditions that is **not** the definition for X, but is logically equivalent to it.
  - If P characterizes X, then every object that satisfies P must be X.

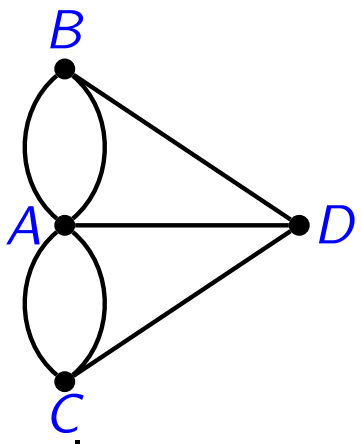
Euler's theorem

Euler's theorem is a chacterization of connected graphs that contain Eulerian paths, and connected graphs that contain Eulerian cycles.

Theorem: (**Euler's theorem**) Let G be a connected undirected graph.

- G has an Eulerian cycle if and only if every vertex $v \in V(G)$ has even degree.
- G has an Eulerian path if and only if there are either zero or two vertices $v \in V(G)$ with odd degrees.

Example: The "Seven Bridges of Königsberg" graph has no Eulerian paths.



- $\deg(A) = 5$, $\deg(B) = 3$, $\deg(C) = 3$, $\deg(D) = 3$.
- This graph has four vertices with odd degrees.

Remark: If G is a connected undirected graph with exactly two vertices u, v in $V(G)$ with odd degrees, then any Eulerian path in G must either be a trail from u to v, or a trail from v to u.

- *Why can't we start our Eulerian path from another vertex?*
- Is it possible for a graph to have exactly one vertex with odd degree?

Intuition for Euler's theorem

Let $G = (V, E, \phi)$ be a connected undirected graph.

[Part 1]: Let $\mathbf{p}$ be an Eulerian path from u to v, and let x be any vertex that is not u or v. Then the degree of x must be even.

(Euler only proved this part, and not its converse, but we still name the theorem after him.)

Idea: To visit x, we must traverse an edge **immediately before** visiting x, and we must traverse an edge **immediately after** visiting x.

- We start with vertex u, so we must traverse an edge **immediately after** visiting u for our first time. To visit u subsequently, we must traverse an edge **immediately before** visiting u, and we must traverse an edge **immediately after** visiting u.
- We end with vertex v, so we must traverse an edge **immediately before** visiting v for our last time. To visit v for a non-last time, we must traverse an edge **immediately before** visiting v, and we must traverse an edge **immediately after** visiting v.

Intuition: For every vertex x (not u or v), there is always a "before" edge and an "after" edge; these edges come in pairs for each time we visit x.

Intuition for Euler's theorem (continued)

Let $G = (V, E, \phi)$ be a connected undirected graph. Suppose there are either zero or two vertices in G with odd degrees.

[Part 2]: Construct an Eulerian path $\mathbf{p} = (v_0, e_1, v_1, e_2, \ldots, e_k, v_k)$ in G.

Idea 1: Select the first vertex v_0.

- If there are two vertices in G with odd degree, then we must select either of them to be the first vertex v_0 of $\mathbf{p}$.
- If instead there are zero vertices in G with odd degree, then G must have an Eulerian cycle, hence any vertex can be the first vertex v_0 in $\mathbf{p}$.

Definition: Let $G = (V, E, \phi)$ be a **connected** undirected graph. An edge $e \in E$ is called a bridge if the deletion of e from G yields a graph that is no longer connected.

- e is a bridge if $G' = (V, E \backslash \{e\}, \phi|_{E \backslash \{e\}})$ is not connected.

Idea 2: We do not want to traverse edges that make us "get stuck".

- When we visit a non-last vertex v_i (i.e. $v_i \neq v_k$), we do not want to traverse a bridge unless absolutely necessary.
  - If at v_i there are two untraversed edges, a bridge e and a non-bridge e', then traversing e immediately after visiting v_i would mean we are unable to traverse e' subsequently.

Introduction to Fluery's algorithm

An algorithm that takes as its input a connected undirected graph G that has an Eulerian path, and returns as its output an Eulerian path.

- **Note:** Fleury's algorithm iteratively modifies G. (G is "destroyed".)
- **Note:** Fleury's algorithm was proposed in 1883.
  - Algorithms pre-dating the first computer are not that rare.
  - Humans were already thinking algorithmically since 300 BC.

Idea of Fluery's algorithm: For constructing an Eulerian path **p**.

```
x ← a suitable vertex v_0 of G as starting vertex for an Eulerian path in G
p ← [x]
while G has at least one edge do
    if x is incident to at least one non-bridge then
        e ← any randomly selected non-bridge incident to x
        x ← the other endpoint of e
    else
        e ← the bridge incident to x (Note: deg(x) must equal 1 in this else-case.)
        x ← the other endpoint of e
    Append p by e and x
    Delete from G the edge e and any (new) isolated vertices (after deleting e)
return p
```

Summary

- Walks, trails, paths
- Eulerian paths, Eulerian cycles
- Euler's theorem
- Introduction to Fleury's algorithm